RA26-Challenge 5: Enunciado Entrenamiento

Entity Manager, sistemas, sprites y escenas

CHASTEROIDS 🚀

Una nave abajo y asteroides que caen. Tú te mueves, disparas, destruyes asteroides, recoges monedas y esquivas lo que se te viene encima. Es un *Mini-Asteroids*, y vas a programar las piezas esenciales de la arquitectura ECS (Entidad-Componente-Sistema): un **gestor de entidades**, dibujo con **sprites** y las **escenas** menú, juego y *game over*.

Todos los ejercicios son rutinas del juego. Juntas forman un juego completo: cuando termines, el juego funciona con tu código.

> ⓘ **Trabaja como en clase**
>
> 1. **En papel.** Piensa, dale vueltas, intenta, falla, prueba de nuevo.
> 2. **En GBTelera**. Copia el código, ejecuta, depura, haz funcionar el juego.
>
> El juego funciona completo. Podrás comprobar ***cada ejercicio individualmente*** si funciona o no, dentro del juego.

Objetivos de aprendizaje

De repaso:

- `call` / `ret` y devolver flags (`Z` / `NZ` y `C` / `NC`)
- `push` / `pop` para conservar registros
- Resta: valor absoluto (`cpl` + `inc`)
- Leer y corregir código: etiquetas, memoria, saltos condicionales, aritmética, mayor y menor

Nuevos:

- **Gestor de entidades**: array de entidades de tamaño fijo, reservar (`alloc`), liberar y **recorrer** (`for_each`).
- **Llamada dinámica** (simular `call hl`): punteros a función.
- **Byte de componentes (bit flags)**: qué tiene cada entidad (`SPRITE` , `PHYSICS` , `COLLIDER` ...).
- **Sistemas en 2 rutinas**: `_one` (una entidad) y `_update` (todas, con `for_each`).
- **Sprites**: la OAM y la **OAM sombra** en WRAM (`shadow_oam`).
- **Colisiones**: uno vs todos, filtrando por tipo con máscara.
- **Escenas**: `init` y `update`
- **Joypad**: Detectar el ***flanco*** (momento en que se pulsa).
- **Módulos y responsabilidades**: cada módulo es el único que escribe sus datos.
- **Velocidades con signo** en 8 bits y comparaciones sin signo para detectar que algo sale de la pantalla.

Entrena para el challenge

Este challenge de entrenamiento tiene **12 ejercicios**. En clase serán los mismos, con sus adaptaciones: el juego, la estructura del proyecto y sus funciones no cambiarán.

Lo importante es que sepas desarrollarlos por ti mismx. Si puedes crear de cero tu propia solución, comprendiendo lo que haces, sabrás adaptarla sin problema.

ⓘ Dos formatos de ejercicio

- **Completar**: la rutina viene con parte del código escrito. Escribe lo que falta en `<<YOUR CODE>>`
- **Corregir**: el código completo viene **con errores de todo tipo**. Debes encontrarlos y arreglarlos.

ⓘ Tiempo orientativo

Ej.	Rutinas	Minutos
1	`man_entity_alloc`	~40
2	`man_entity_for_each`	~45
3	`sys_physics_one`, `sys_physics_update`	~50
4	`sys_shot_spawn`	~40
5	`sys_shot_update`	~35
6	`sys_prerender_one`	~40
7	`sys_prerender_update`	~25
8	`sys_render_update`	~25
9	`sys_collision_overlaps`	~30
10	`sys_collision_one`	~40
11	`scene_menu_update`, `scene_gameover_update`	~30
12	`scene_game_update`, `game_handle_collisions`	~40
	Total	**~7.5 h**

Para equipos de 4-5 personas, **1,5-2,0 horas/persona** de entrenamiento básico.
Doble para entrenamiento completo. Mejor 1h/día máximo.

El juego

- ◄ ► mueven la nave (`INPUT`), **A** dispara (solo **un disparo** a la vez), **START** empieza y vuelve al menú.
- Caen asteroides, rectos o en diagonal, y las diagonales apuntan a la zona donde puedes estar.
- Un disparo que alcanza un asteroide lo destruye (**+1 punto**) y a veces suelta una **moneda** con un `+`. Si la recoges, **+5**.
- Si un asteroide te toca, **explotas**: la partida acaba tras la explosión y aparece *Game Over* sobre el último fotograma.

Organización del proyecto

Carpetas por tipo: `man/` (gestores), `sys/` (sistemas), `scn/` (escenas), `engine/` (servicios), `ui/`, `gfx/`.

```
src/
├── main.asm                Bucle principal, y líneas DEF que eliges (TEST, EJ01…EJ12)
├── engine/                 Pantalla, memoria, texto, joypad, aleatorios (dado)
├── gfx/                    Tiles y fondo
├── man/
│   ├── man_entity.asm      Gestor de entidades: init, alloc, create, free, for_each
│   └── man_scene.asm       Gestor de escenas: scene_change, scene_update
├── sys/                    Sistemas: un fichero cada uno
│   ├── sys_physics.asm     Ej. 3       sys_shot.asm        Ej. 4, 5
│   ├── sys_prerender.asm   Ej. 6, 7    sys_render.asm      Ej. 8
│   ├── sys_collision.asm   Ej. 9, 10   sys_input / sys_spawn / sys_selfdestroy (dados)
├── scn/                    Escenas
│   ├── scn_menu.asm        Ej. 11      scn_gameover.asm    Ej. 11
│   ├── scn_game.asm        Ej. 12      scn_game_rules.asm  Ej. 12 (y reglas dadas)
├── ui/                     Marcador y botón Start (dados)
├── chcode/                 Pruebas y código del proyecto (no tocar)
└── ej01.asm … ej12.asm     ← TU CÓDIGO: un fichero por ejercicio (en el proyecto)
```

ⓘ Cómo se integra tu código

Ejemplo:

```
sys_physics_one::
    jp ej03   ;; Salto a tu código
```

Tú implementas `ej03::` en `ej03.asm` y tu código se ejecuta cuando el juego llame a `sys_physics_one::`

Escenas

Hay tres: **menú**, **juego** y **game over**. Cada una son dos rutinas: `init` (una vez al entrar, con la pantalla apagada) y `update` (una vez por fotograma). `scene_change` apaga la pantalla esperando al VBLANK, ejecuta el `init` y la enciende. Dibujar con la pantalla encendida puede dañar una pantalla real.

Renderizar en dos pasos

La OAM solo se puede escribir durante el **VBLANK**. Para hacerlo rápidamente, usamos dos pasos:

1. **Pre-render** (`sys_prerender_update`): copia los 4 bytes `Y, X, TILE, ATTR` de cada entidad a la **OAM sombra** (`shadow_oam`), una copia de la OAM en WRAM.
2. **Render** (`sys_render_update`): espera al VBLANK y copia la `shadow_oam` a la OAM.

Entidades, componentes y sistemas

Una **entidad** es un objeto de **8 bytes** en el array de entidades. Esta es la estructura de esos 8 bytes:

Offset	Campo	Para qué sirve	Componente
0	`CMP`	Qué componentes tiene (***bitflags***). `CMP = 0` → **Espacio libre**	
1	`Y`	Posición vertical	`SPRITE`
2	`X`	Posición horizontal	`SPRITE`
3	`TILE`	ID de tile (sprite) de la entidad	`SPRITE`
4	`ATTR`	Atributos del sprite	`SPRITE`
5	`VY`	Velocidad vertical (con signo)	`PHYSICS`
6	`VX`	Velocidad horizontal (con signo)	`PHYSICS`
7	`KIND`	De qué tipo es (***bitflags***)	`COLLIDER`

El byte `CMP` indica qué componentes usa la entidad y `KIND` de qué tipo es. Ambos se organizan en ***bitflags***:

```
 bit   7 6 5 4 3 2 1 0              bit   7 6 5 4 3 2 1 0
 CMP  |1|x|x|1|0|1|1|1|            KIND  |x|x|x|x|0|0|1|0|
ALIVE ─┘     | | | | └─ SPRITE       KIND_BONUS ─┘ | | └─ KIND_SHIP
      INPUT ─┘ | | └── PHYSICS        KIND_ENEMY ─┘ └── KIND_SHOT
  SELFDESTROY ─┘ └──── COLLIDER
```

Así, el bit `ALIVE` indica que la entidad está viva, y el bit `PHYSICS` que le afectan las físicas. El bit `KIND_ENEMY` indica que es un enemigo y se usa para decidir con qué colisiona.

Cada **sistema** procesa las entidades que tienen los componentes (datos) que el sistema necesita, con dos rutinas:

- `sys_xxx_one` : procesa **UNA** entidad (dirección en `DE`).
- `sys_xxx_update` : procesa **todas**. Carga en `HL` la dirección de `sys_xxx_one` y salta a `man_entity_for_each` , que llamará a `sys_xxx_one` una vez por cada entidad viva.

Las **reglas del juego** (qué pasa tras una colisión, cuántos puntos...) no están en los sistemas: están en `scn/scn_game_rules.asm` . Los sistemas son generales: no conocen los detalles del juego, solo de las entidades.

> ⓘ **Principio de *RESPONSABILIDAD***
> Los datos de un módulo (por ejemplo, `shot_entity`) solo los escribe ese módulo. Los demás usan sus rutinas. Por eso `shot_spawn` (una regla del juego) no toca `shot_entity` : calcula dónde aparece el disparo y llama a `sys_shot_spawn` , que es quien lo crea y lo anota.

> **Almacenar direcciones en memoria (punteros)**
> Un puntero de 16 bits (una dirección de memoria) se guarda con el **byte bajo primero** (*little-endian*).
> Si está en `HL` , guardas `L` y luego `H` .

Plantillas de entidades

Crear una entidad a mano exige escribir sus 8 campos. Para evitarlo, el juego guarda en la ROM una **plantilla** por tipo (`ShotTemplate` , `AsteroidTemplate` ...): los 8 bytes con los valores iniciales. `man_entity_create` reserva

una entidad y copia la plantilla en ella. Después solo cambias lo que depende de la situación, como la posición. Es la idea de los tiles del Challenge 4: datos en la ROM, copiados con `memcpy8`.

Datos

Dato	Bytes	Qué guarda	La escribe
`entity_array`	64	Las 8 entidades. Alineado (dirección `$xx00`)	`man_entity`
`shadow_oam`	160	Copia de la OAM, en WRAM	pre-render
`shadow_cursor`	2	Dónde va el próximo sprite en `shadow_oam`	`sys_prerender`
`shot_entity`	2	Dirección de la entidad del disparo (0 = no hay)	`sys_shot`
`collision_probe`	2	La sonda en curso	`sys_collision_update`
`collision_mask`	1	Los tipos con los que choca la sonda	`sys_collision_update`
`collision_hits`	1	Colisiones de esta pasada	`sys_collision_update`, `sys_collision_one`
`input_held` / `input_pressed`	1 + 1	Botones pulsados / recién pulsados	`input_update` (tú solo lees)

Un fotograma

```
main (cada fotograma)
  input_update       lee el joypad (mantenido y recién pulsado)
  frame_tick
  scene_update       el update de la escena actual
  sys_render_update  espera a VBLANK y copia los sprites   ← Ej. 8
  hud_update, fx_update   (aún en VBLANK)
```

`scene_game_update` (Ej. 12) decide el orden de los sistemas:

```
  ¿la nave ya no existe?  → Game Over
  1. game_handle_collisions      las colisiones, con lo que se acaba de dibujar
  2. sys_input_update            la cruceta pone el VX
     game_handle_shooting        A dispara
     sys_spawn_update            salen asteroides
     sys_physics_update          mueve a todos (la nave, los asteroides y el disparo)
     sys_shot_update             ¿el disparo sigue ahí? Si no, lo olvida
     sys_selfdestroy_update      las explosiones se acaban
  3. sys_prerender_update        prepara los sprites
```

Comprueba cada ejercicio

Cada ejercicio tiene dos casos de prueba, **P1 y P2** (como en el Challenge 3).

1. En `main.asm`: `DEF TEST EQUS "test_ej05"`. Ejecuta `make` y abre `game.gb` en BGB.
2. BGB se para en cada `ld b, b`. Compara CPU y MEM con el RESULTADO ESPERADO del final de `ej05.asm`.
3. Las pruebas usan las rutinas del proyecto para todo lo demás: un fallo en otro ejercicio no afecta al tuyo.

Para probarlo dentro del juego: `DEF TEST EQUS "game"` y `DEF EJ05 = 1`. Con `0`, el juego usa la rutina del proyecto. Puedes poner a `1` todos los que quieras.

En clase no habrá rutinas del proyecto para el juego: funcionará cuando estén todos los ejercicios. Si funciona, **bonus**.

CHALLENGE

Código común

⚠ **Este código es común para todos los ejercicios. Tus soluciones deben usarlo.**

```
IF !DEF(MAN_ENTITY_INC)
DEF MAN_ENTITY_INC = 1
;;-----------------------------------------------------------------
;; Entities: array of MAX_ENTITIES entities of SIZEOF_ENTITY bytes
;;-----------------------------------------------------------------
DEF MAX_ENTITIES = 8
RSRESET
DEF E_CMP         RB 1          ;; 0  Which components the entity has (bitflags)
DEF E_SPRITE      RB 1          ;; SPRITE data (offset): same OAM entry 4 bytes
DEF E_Y =     E_SPRITE          ;; 1  ┐ E_Y = E_SPRITE (same byte)
DEF E_X           RB 1          ;; 2  │ SPRITE bytes can be copied as they are.
DEF E_TILE        RB 1          ;; 3  │ Y and X are OAM coordinates (screen + 16, + 8)
DEF E_ATTR        RB 1          ;; 4  ┘
DEF E_VY          RB 1          ;; 5  ┐ PHYSICS: velocity (signed) in pixels per frame
DEF E_VX          RB 1          ;; 6  ┘
DEF E_KIND        RB 1          ;; 7  COLLIDER: what kind of thing this entity is
                                ;;    SELFDESTROY: frames left (shared byte: an entity
                                ;;    with it has no COLLIDER. Avoids a 9th byte)
DEF SIZEOF_ENTITY RB 0          ;; = 8
DEF ENTITY_ARRAY_SIZE = MAX_ENTITIES * SIZEOF_ENTITY

;; Bits of CMP. CMP = 0 → the entity is FREE.
;; Changing the bits while the game runs changes what the entity does
DEF CMP_SPRITE_BIT      = 0
DEF CMP_PHYSICS_BIT     = 1
DEF CMP_COLLIDER_BIT    = 2
DEF CMP_SELFDESTROY_BIT = 3
DEF CMP_INPUT_BIT       = 4
DEF CMP_ALIVE_BIT       = 7
;; Bit Masks for CMPs
DEF CMP_SPRITE          = 1 << CMP_SPRITE_BIT
DEF CMP_PHYSICS         = 1 << CMP_PHYSICS_BIT
DEF CMP_COLLIDER        = 1 << CMP_COLLIDER_BIT
DEF CMP_SELFDESTROY     = 1 << CMP_SELFDESTROY_BIT
DEF CMP_INPUT           = 1 << CMP_INPUT_BIT ;; Controlled by the player. It has no data
DEF CMP_ALIVE           = 1 << CMP_ALIVE_BIT

ENDC
```

```
IF !DEF(HARDWARE_INC)
DEF HARDWARE_INC = 1
;;-----------------------------------------------------------------
;; Registers
;;-----------------------------------------------------------------
DEF rP1    = $FF00              ;; Joypad
```

```
DEF rDIV   = $FF04              ;; Divider (changes all the time)
DEF rLCDC  = $FF40              ;; LCD control. Bit 7: LCD on/off
DEF rLY    = $FF44              ;; Line being drawn (144-153: VBLANK)
DEF rBGP   = $FF47              ;; Background palette
DEF rOBP0  = $FF48              ;; Sprite palette 0
DEF rOBP1  = $FF49              ;; Sprite palette 1 (sprites with ATTR bit 4)

DEF VBLANK_LINE   = 144
DEF LCDC_ON_BIT   = 7
DEF LCDC_VALUE    = %10010011   ;; LCD on · tiles at $8000 · sprites on · background on
DEF PALETTE       = %00011011   ;; colour 0 = black ... colour 3 = white

DEF VRAM_TILES    = $8000
DEF TILEMAP       = $9800
DEF TILEMAP_WIDTH = 32
DEF OAM_START     = $FE00
DEF OAM_SIZE      = 160         ;; 40 sprites x 4 bytes
DEF WRAM_END      = $E000

;; How the 40 OAM SPRITE slots are used:
;;   slots  0-7   the entities (copied from shadow_oam every frame)  sys/sys_render.asm
;;   slots  8-15  the start button (written once) ui/start_button.asm
;;   slots 16-39  unused (hidden)
;;-----------------------------------------------------------------
;; Buttons (input_held / input_pressed). 1 = pressed
;;-----------------------------------------------------------------
DEF BTN_RIGHT_BIT  = 0
DEF BTN_LEFT_BIT   = 1
DEF BTN_UP_BIT     = 2
DEF BTN_DOWN_BIT   = 3
DEF BTN_A_BIT      = 4
DEF BTN_B_BIT      = 5
DEF BTN_SELECT_BIT = 6
DEF BTN_START_BIT  = 7
DEF BTN_A          = 1 << BTN_A_BIT
DEF BTN_START      = 1 << BTN_START_BIT

ENDC
```

```
IF !DEF(GAME_INC)
DEF GAME_INC = 1

;; KIND (one bit per kind, so a mask can select several)
DEF KIND_SHIP  = %0001
DEF KIND_SHOT  = %0010
DEF KIND_ENEMY = %0100
DEF KIND_BONUS = %1000

;; The ship is always the first entity created after man_entity_init
DEF SHIP_ENTITY EQUS "entity_array"
;;-----------------------------------------------------------------
;; Game rules (all in OAM coordinates: screen y + 16, screen x + 8)
;;-----------------------------------------------------------------
DEF SHIP_Y        = 144     ;; screen y = 128
DEF SHIP_X_START  = 84
DEF INPUT_SPEED   = 2       ;; pixels per frame of an entity with INPUT
```

```
DEF INPUT_X_MIN    = 8       ;; an entity with INPUT cannot go past these limits
DEF INPUT_X_MAX    = 160     ;; (sys_physics keeps it inside)
DEF SHOT_SPEED     = 4       ;; pixels per frame (the shot goes up: VY = -SHOT_SPEED)
DEF ENTITY_OUT_Y   = 160     ;; Y from here on → off the bottom
DEF ENTITY_OUT_X   = 168     ;; X from here on → off the right
                             ;; (a coordinate that goes below 0 wraps to 255+)
DEF HIT_SIZE       = 7       ;; two sprites overlap if |dY| < 7 and |dX| < 7
DEF EXPLOSION_FRAMES = 12    ;; frames an explosion stays on the screen
DEF DIAGONAL_VY    = 2       ;; diagonal asteroids always fall at this speed (VX = +/-1)
DEF DIAGONAL_SHIFT = SHIP_Y / DIAGONAL_VY  ;; so, from the top to the ship's row
                                           ;; they move this much sideways (72)
DEF DIAGONAL_SPLIT = INPUT_X_MIN + DIAGONAL_SHIFT ;; a target from here on:
                                                  ;; it comes from the left (80)
DEF SIDE_SPAWN_X_LEFT  = 0  ;; diagonals can also come in from the sides of the screen:
DEF SIDE_SPAWN_X_RIGHT = ENTITY_OUT_X - 1 ;; just outside (X=0→hidden, 167→one column)
DEF SIDE_SPAWN_MIN_Y   = 16 ;; from the top of the screen (OAM Y of the screen row 0)..
DEF SIDE_SPAWN_STEPS   = 37 ;; ..in steps of 2, down to half the screen (Y = 88)
DEF SPAWN_FIRST_FRAMES = 45
DEF SPAWN_MIN_FRAMES   = 30 ;; next asteroid in 30 + (0..31) frames
DEF POINTS_ENEMY   = 1       ;; points for an asteroid (one digit: 0-9)
DEF POINTS_BONUS   = 5       ;; points for a bonus item (one digit: 0-9)

ENDC
```

Rutinas ya resueltas

⚠ **Estas rutinas ya funcionan. Úsalas tal cual: importa su contrato, no su código.**

Contrato: 📥 entrada · ⬅ salida · ❌ registros que modifica

Rutina	Qué hace	Contrato
`man_entity_init`	Libera todas las entidades	❌ `A`, `B`, `HL`
`man_entity_free`	Libera una entidad	📥 `DE` = entidad ❌ `A`
`man_entity_create`	Reserva una entidad y copia una **plantilla** en ella	📥 `HL` = plantilla ⬅ `Z` y `HL` = entidad, o `NZ` si no hay hueco ❌ `A`, `BC`, `HL`. **NO** `DE`
`call_hl`	Salta a la dirección que hay en `HL` (hace de `call hl`)	
`memcpy8`	Copia `B` bytes	📥 `HL` origen, `DE` destino, `B` bytes (1-255) ⬅ `HL` y `DE` han avanzado `B` bytes ❌ `A`, `B`, `DE`, `HL`
`memset8`	Rellena `B` bytes	📥 `HL` destino, `A` valor, `B` bytes (1-255) ❌ `B`, `HL`
`wait_vblank`	Espera al VBLANK	❌ `A`, `HL`

Rutina	Qué hace	Contrato
`scene_change`	Cambia de escena (apaga la pantalla, ejecuta el `init`, la enciende)	📥 `HL` = escena (`scene_game`, `scene_menu`, `scene_gameover`) ❌ `A`, `BC`, `DE`, `HL`
`sys_input_update`	Pone el `VX` de las entidades con `INPUT` según la cruceta	❌ `A`, `BC`, `DE`, `HL`
`game_handle_shooting`	Si se acaba de pulsar A y la nave obedece, dispara	❌ `A`, `BC`, `DE`, `HL`
`shot_spawn`	Calcula dónde aparece el disparo (sobre la nave) y llama a `sys_shot_spawn`	❌ `A`, `BC`, `DE`, `HL`
`sys_shot_destroy`	Quita el disparo: libera su entidad y pone `shot_entity` a 0	❌ `A`, `DE`
`sys_shot_get`	Devuelve el disparo	⬅ `HL` = entidad del disparo; `Z` si no hay ❌ `A`, `HL`
`sys_physics_limit_x`	Mantiene a una entidad con `INPUT` dentro de los límites	📥 `HL` = `&X`, `B` = nueva `X` ❌ `A`
`sys_collision_update`	Una **sonda** contra todas las entidades	📥 `HL` = sonda, `B` = máscara de tipos ⬅ `A` = cuántas chocaron (ya liberadas); `Z` si ninguna ❌ `A`, `BC`, `DE`, `HL`
`is_ship_gone`	¿Ha desaparecido la nave?	⬅ `Z` si sí ❌ `A`
`game_enemy_hit`	El disparo ha destruido asteroides	📥 `A` = cuántos ❌ `A`, `BC`, `DE`, `HL`
`game_ship_destroyed`	La nave choca: se convierte en explosión	❌ `A`
`game_bonus_taken`	La nave recoge una moneda	❌ `A`, `HL`
`sys_spawn_update`, `sys_selfdestroy_update`, `sys_shot_init`...	Aparición de asteroides, fin de las explosiones...	

> ⚠ **Cada ejercicio se resuelve aislado**
> Si un ejercicio usa una rutina de otro ejercicio, asume que existe y cumple **exactamente** su contrato.

> 🔥 `entity_array` **está alineado**
> Asume que `entity_array` se almacena ***alineado*** (`ALIGN[8]`). Su dirección es `$xx00` (primer byte). La parte alta (`xx`) no cambia para ninguno de sus elementos.

Ejercicio 1

🐞 `man_entity_alloc` : Encuentra y corrige los errores

Reserva una entidad libre y devuelve `Z` con `HL` apuntando a ella. Si el array está lleno, devuelve `NZ`.

🔥 Consejos

- Lee el contrato: qué recibe, qué devuelve, qué puede modificar.
- Sigue el código con un caso: entidad 0 viva y entidad 1 libre.
- Revisa cada instrucción: ¿existe?, ¿hace lo que debe?

⁝☰ Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, hay un hueco.** Entidades 0 y 1 vivas, la 2 libre. A=SPRITE | PHYSICS, DE=1234 (no cambia).
- **P2, array lleno.** Las 8 entidades están vivas. A = SPRITE, DE = 1234.

⚠ Qué entregas

Indica línea y error, y escribe al lado su corrección. En el proyecto: código corregido, con fixes indicados.

Tipos de error: **N** (instrucción que no existe), **E** (errónea), **S** (sintaxis), **F** (falta algo).

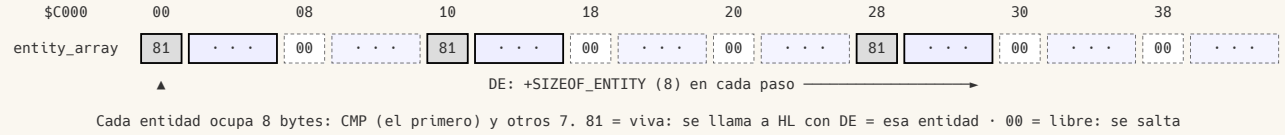
```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Looks for a free entity and reserves it.
;; 📥 INPUT:   A: components the new entity will have (CMP_SPRITE, ...)
;; 🡸 RETURNS: Z,  HL = address of the entity, and [HL] = A | CMP_ALIVE
;;             NZ, there is no free entity
;; ❌ May modify A, BC, HL. Must NOT modify DE
;; ⚠️ It does not change the other fields: the caller fills them in
man_entity_alloc::
    ld   hl, entity_array
    ld   b, ENTITY_ARRAY_SIZE
.next
    bit  CMP_ALIVE_BIT, [hl]
    jr   nz, .found
    ld   a, l
    add  hl, SIZEOF_ENTITY
    ld   l, a
    dec  b
    jr   nz, .next
    or   1
    ret
.found
    ld   a, c
    ld   [hl], a
    or   1
    ret
```

Ejercicio 2

Completa `man_entity_for_each`

Recorre las entidades; para cada una viva, llama a la dirección `HL` (`"call hl"`). Al llamar a `HL` , `DE` = dirección de la entidad viva.

Así, `HL` será la dirección del `sys_xxx_one` de cualquier sistema. Este mismo bucle se reutiliza para que todos los sistemas procesen todas las entidades vivas una a una.



Cada entidad ocupa 8 bytes: CMP (el primero) y otros 7. 81 = viva: se llama a HL con DE = esa entidad · 00 = libre: se salta

Pasos (intenta antes sin mirarlos)

1. Guarda con `push` lo que la rutina puede cambiar: `BC` (tu contador), `DE` (la entidad) y `HL` (la rutina).
2. Llama con `call call_hl` . `DE` ya apunta a la entidad.
3. Restaura con `pop` , en orden inverso.
4. En `.skip` : suma `SIZEOF_ENTITY` a `E` (el array está alineado), resta 1 a `B` y repite mientras no sea 0. Al final, `ret` .

Consejos

- La rutina llamada (dirección en `HL`) podría modificar cualquier registro.
- Cada `push` necesita su `pop` . Si falta uno, la pila se descuadra.
- Para inspeccionar: pon `ld b, b` (BGB se detiene) y mira los registros.

Si funciona

- Casi todo depende de este ejercicio: si falla, no se dibuja ni se mueve nada.
- Si dibuja y se cuelga, revisa `push` y `pop` .

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, 3 entidades vivas.** Entidades 0, 2 y 5 vivas (solo CMP = 81). HL = tst_cb_log: apunta en tst_log el byte bajo de cada entidad y destruye todos los registros.
- **P2, 8 entidades vivas.** Las 8 entidades vivas. La rutina llamada destruye BC, DE y HL en cada llamada: for_each debe seguir bien.

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Calls a routine once for every entity that is alive.
;; 📥 INPUT: HL: address of the routine to call
;;           The routine receives DE = the entity, and it may modify
;;           every register: for_each looks after its own state.
;; ❌ May modify A, BC, DE, HL
;; ⚠️ Use "call call_hl" to call the routine whose address is in HL
man_entity_for_each::
    ld   de, entity_array       ;; \ DE = first entity
    ld   b, MAX_ENTITIES        ;; / B = entities left

.next
    ld   a, [de]                ;; \ A = entity.CMP
    bit  CMP_ALIVE_BIT, a       ;; | Is it alive?
    jr   z, .skip               ;; / No (it is free): skip it

    ;; <<YOUR CODE>>
.skip                ;; .skip is required by line 15
    ;; <<YOUR CODE>>
```

Ejercicio 3

🐞 `sys_physics_one`, `sys_physics_update`: **Encuentra y corrige los errores**

Este código mueve ***UNA*** entidad (`Y += VY`, `X += VX`) y decide qué pasa en los bordes.
Con `INPUT` salta a `sys_physics_limit_x`. Sin `INPUT`, se libera al salir de la pantalla.
`sys_physics_update` lo aplica a todas las entidades.

🔥 Consejos

- Una velocidad negativa va en complemento a 2: `VX = -1` es `$FF`. Sumarla resta 1.
- Una `X` que baja de 0 pasa a 255: `cp ENTITY_OUT_X` es sin signo.
- `cp` deja el carry a 1 si `A` es menor.
- Sigue el código con una entidad concreta y anota los registros.

☰ Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, un asteroide se mueve.** Entidad 1: asteroide en Y=50, X=80, VY=2, VX=-1 (FF). DE = entidad 1.
- **P2, sale por la izquierda.** Entidad 1: asteroide en Y=50, X=0, VY=1, VX=-1 (FF). DE = entidad 1.

⚠ Qué entregas

Indica línea y error, y escribe al lado su corrección. En el proyecto: código corregido, con fixes indicados.
Tipos de error: **N** (instrucción que no existe), **E** (errónea), **S** (sintaxis), **F** (falta algo).

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Moves ONE entity: Y += VY, X += VX. Then, what happens at the edges depends on it:
;;  - The entities with INPUT (the ship) are controlled by the player: they
```

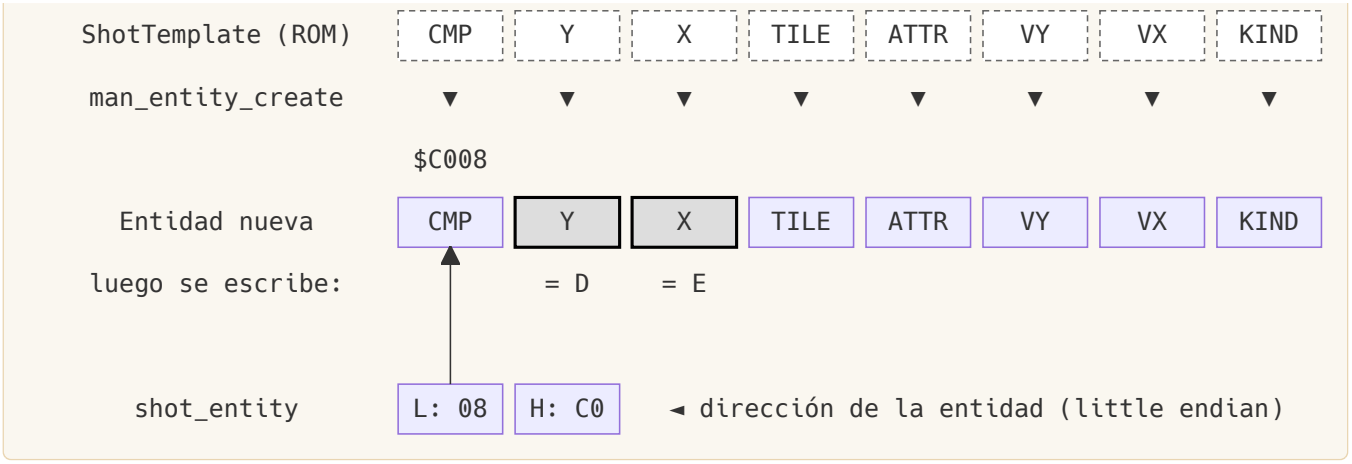
```
;;    cannot leave. Their limits are done by sys_physics_limit_x (given): jump to it.
;;  - The others are freed if they leave the screen (Y >= ENTITY_OUT_Y or
;;    X >= ENTITY_OUT_X). Note: a coordinate that goes below 0 wraps to 255.
;; Only works on entities with the PHYSICS component.
;; (A semi-generic system: ideally every entity would have its own limits
;;  and its own policy, but that is too much for this level.)
;; 📥 INPUT: DE: entity
;; ❌ May modify A, BC, DE, HL
sys_physics_one::
    ld   a, [de]
    and  CMP_PHYSICS
    ret  nz
    ld   hl, E_VY
    add  hl, de
    ld   a, [hl+]
    ld   c, a
    ld   b, [hl]
    ld   hl, E_Y
    add  hl, de
    ld   a, [hl]
    add  c
    ld   [hl+], c
    ld   c, a
    ld   a, [hl]
    sub  b
    ld   [hl], a
    ld   a, [de]
    and  CMP_INPUT
    jp   nz, sys_physics_limit_x
    ld   a, c
    cp   ENTITY_OUT_Y
    jr   nc, .leaves
    ld   a, b
    cp   ENTITY_OUT_Y
    ret  nc
.leaves
    jp   man_entity_free
```

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Moves ALL the entities that have PHYSICS.
;; ❌ May modify A, BC, DE, HL
sys_physics_update::
    ld   hl, sys_physics_one
    jp   man_entity_for_each
```

Ejercicio 4

Completa `sys_shot_spawn`

Crea ***EL*** disparo llamando a `man_entity_create` y sitúalo en `(D, E) = (Y, X)`. Solo puede haber un disparo: si ya existe o no hay entidades libres, no hace nada y devuelve `NZ`. Si lo crea, anota su dirección en `shot_entity` y devuelve `Z`.



ⓘ Pasos (intenta antes sin mirarlos)

1. Carga `ShotTemplate` en `HL` y llama a `man_entity_create`.
2. Comprueba si se ha creado. Vuelve si no.
3. Guarda `HL` (dirección de la entidad) en `shot_entity`: primero `L`, luego `H`.
4. Escribe los bytes `(E_Y, E_X)` de la entidad con `(D, E)`.
5. Devuelve `Z`.

🔥 Consejos

- `man_entity_create` conserva `DE` (no lo modifica).
- `ShotTemplate` tiene todos los datos de un disparo. Solo hay que ajustar la posición inicial `(Y, X)`.

☰ Si funciona

- Pulsa A: sale un disparo de la nave. Si hay disparo, A no crea otro.

☰ Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, no hay disparo.** Solo la nave. shot_entity = 0000. D = 100 (Y), E = 50 (X).
- **P2, ya hay un disparo.** La entidad 1 es un disparo y shot_entity apunta a ella. D = 100, E = 50.

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Creates THE shot at a position. There can only be ONE shot: if it already
;; exists, or there is no free entity, it does nothing.
;; The entity is created from ShotTemplate data, and then Y and X are changed.
;; The system stores the address of the entity in shot_entity
;; Where the shot appears (D, E) = (Y, X) is a rule of the game: shot_spawn decides it.
;; 📥 INPUT:   D = Y, E = X
;; 🔙 RETURNS: Z, the shot has been created
;;             NZ, it has not (there was one, or there is no free entity)
;; ❌ May modify A, BC, HL. Does NOT modify DE
;; ⚠️ Use man_entity_create
sys_shot_spawn::
    ld   a, [shot_entity + 1]   ;; \ A = high byte of shot_entity
```

```
    or   a                      ;; | 0 means there is no shot
    ret  nz                     ;; / There is one already: NZ

    ;; <<YOUR CODE>>
```

Ejercicio 5

Completa `sys_shot_update`

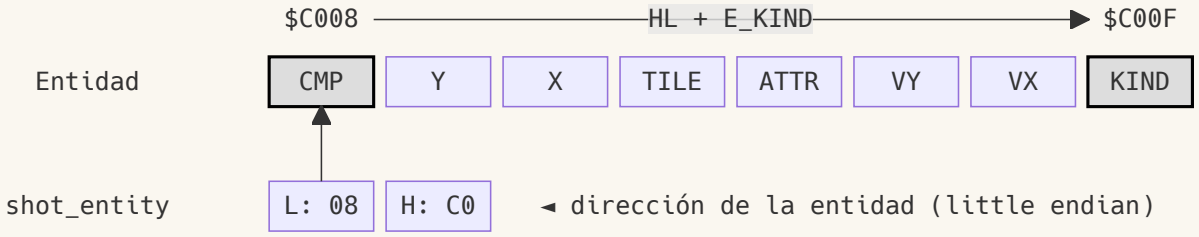
`sys_shot_update` comprueba que ***EL*** disparo sigue ahí. Si ya no está, lo olvida: pone `shot_entity` a 0.

`sys_physics` mueve el disparo y lo elimina si escapa de la pantalla, `sys_collision` detecta si impacta y lo destruye. `sys_shot_update` comprueba si ha sido destruido, para dejar de seguirlo.

El disparo sigue ahí si:

1. Su entidad **está viva** (`ALIVE`)
2. y **es** `KIND_SHOT`

La entidad podría haberse liberado (no `ALIVE`), y podría haber otra entidad nueva (no sería `KIND_SHOT`).



Pasos (intenta antes sin mirarlos)

1. El código dado carga en `HL` el puntero `*(shot_entity)` y sale si no hay disparo.
2. ¿Está viva su entidad? `bit CMP_ALIVE_BIT, [hl]` . Si no, olvídalo.
3. ¿Es aún un disparo? Apunta `HL` a `KIND` (suma `E_KIND` a `L`) y compara con `KIND_SHOT` . Si lo es, `ret` .
4. Olvidarlo: pon a 0 los dos bytes de `shot_entity` y `ret` .

Consejos

- Un puntero de 16 bits (`shot_entity`) se guarda con el byte bajo primero (*little-endian*).
- Puedes usar `bit n, r` para comprobar bits.
- `cp KIND_SHOT` deja `Z` si es igual: `ret z` .

Si funciona

- El disparo sube y desaparece por arriba (lo libera la física). Después puedes disparar otra vez.
- Si no se olvida bien, no podrás disparar más: `shot_entity` se queda marcado.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, el disparo sigue ahí.** Disparo en la entidad 1: Y = 100, X = 60. Llamamos a sys_shot_update.
- **P2, ya no es el disparo.** shot_entity apunta a la entidad 1. Llamamos DOS veces a sys_shot_update: primero con la entidad 1 libre (CMP = 00) y después con un asteroide en su sitio. Tras cada llamada, shot_entity debe valer 0000.

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks that THE shot is still there, and if not, forgets it (shot_entity = 0).
;; sys_physics moves the shot and frees it when it leaves the screen, like any
;; other entity: this system does not move anything. The shot is still there only if
;; its entity is alive AND it is still a shot (KIND_SHOT): its slot may have been
;; freed, or taken by another entity (an asteroid, for example).
;; If there is no shot, it does nothing.
;; ❌ May modify A, HL
sys_shot_update::
    ld   a, [shot_entity]       ;; \ HL = *(shot_entity)
    ld   l, a                   ;; |
    ld   a, [shot_entity + 1]   ;; |
    ld   h, a                   ;; / = the shot

    or   a                      ;; \ Is there a shot?
    ret  z                      ;; / No: nothing to do

    ;; <<YOUR CODE>>
```

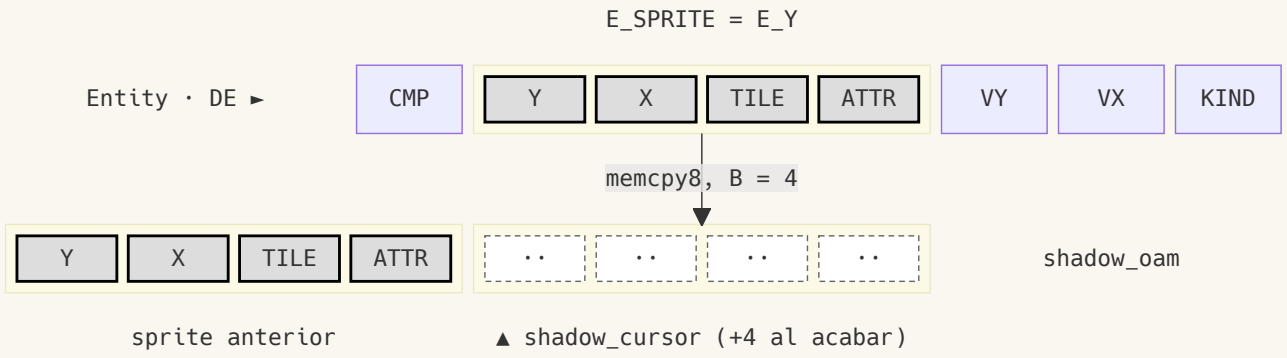
Ejercicio 6

Implementa `sys_prerender_one`

Si la entidad en `DE` tiene `CMP_SPRITE`, copia los 4 bytes del ***sprite*** a `shadow_cursor`, la dirección del siguiente hueco en la ***Shadow OAM***. Después, avanza +4 `shadow_cursor`, quedando listo para copiar el siguiente.

Si no tiene `CMP_SPRITE`, no hace nada.

El ***Sprite*** son los 4 bytes `Y, X, TILE, ATTR` de la entidad:



Pasos (intenta antes sin mirarlos)

1. Filtro: sin `CMP_SPRITE`, `ret`.
2. `HL` = origen: `ld hl, E_SPRITE` y `add hl, de`.
3. `DE` = destino: el valor de `shadow_cursor` (2 bytes, bajo y alto).
4. `B` = 4 y `call memcpy8`. Al volver, `DE` está 4 bytes más allá.

5. Guarda `DE` en `shadow_cursor` y `ret`.

Consejos

- `memcpy8`: `HL` origen, `DE` destino, `B` bytes.
- Calcula `HL` ***ANTES*** de cargar el cursor en `DE`: necesitas `DE` (la entidad) para el origen.

Si funciona

- Se ven la nave y los asteroides. Si no se ve nada, revisa los ejercicios 2, 6, 7 y 8: el dibujo usa los cuatro.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, entidad con sprite.** Entidad 0 (la nave, con SPRITE). shadow_cursor = shadow_oam + 8. La OAM sombra está llena de EE.
- **P2, entidad sin sprite.** Entidad con PHYSICS pero SIN SPRITE. shadow_cursor = shadow_oam + 8. La OAM sombra, llena de EE.

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Prepares ONE entity to be drawn: copies its sprite (Y, X, TILE, ATTR:
;; 4 bytes, in the same order as the OAM) to the shadow OAM, to the place
;; that shadow_cursor says, and moves the cursor 4 bytes on.
;; If the entity does not have CMP_SPRITE, it does nothing.
;; 📥 INPUT: DE: entity
;; ❌ May modify A, B, DE, HL
;; ⚠️ Use memcpy8
sys_prerender_one::
    ;; <<YOUR CODE>>
```

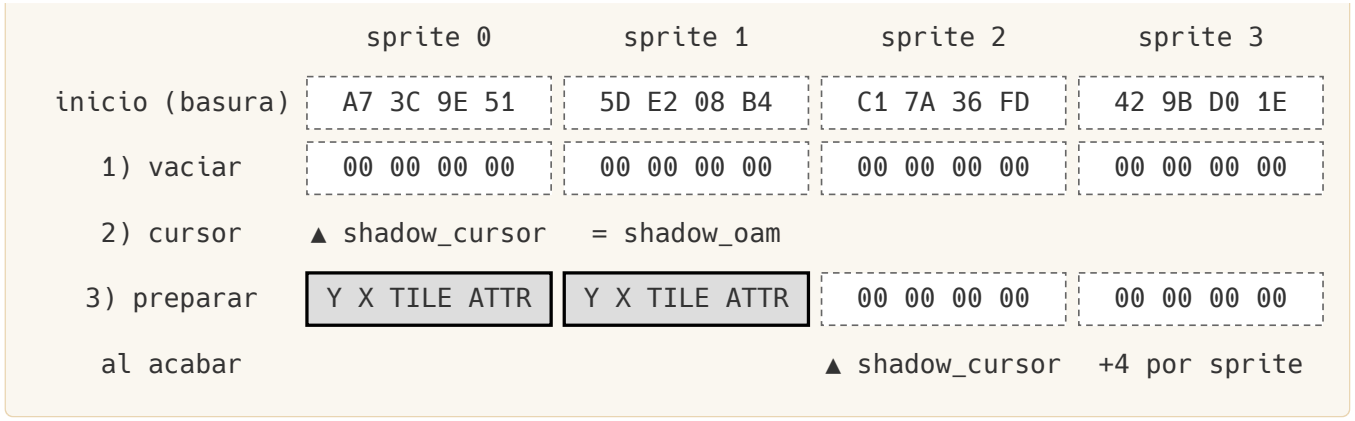
Ejercicio 7

Implementa `sys_prerender_update`

Prepara la `shadow_oam`:

1. La llena de 0's (`SHADOW_OAM_SIZE` bytes). Los sprites quedan con `X=0`, fuera de pantalla.
2. Pone `shadow_cursor` a la dirección de inicio `shadow_oam`
3. Lanza `man_entity_for_each` para que copie las entidades vivas con sprite, usando `sys_prerender_one`

El diagrama ejemplifica cómo queda `shadow_oam` en cada paso:



ⓘ Pasos (intenta antes sin mirarlos)

1. Vacía la OAM sombra: `HL` = `shadow_oam`, `A` = 0, `B` = `SHADOW_OAM_SIZE`, `call memset8`.
2. Pon `shadow_cursor` = dirección de `shadow_oam` (`LOW` y `HIGH`).
3. Carga `sys_prerender_one` en `HL` y salta a `man_entity_for_each`.

Consejos

- `LOW(etiqueta)` y `HIGH(etiqueta)` dan el byte bajo y el alto de una dirección.

Si funciona

- Cuando un asteroide desaparece, su sprite desaparece con él.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, 2 entidades con sprite.** Entidades 0 (nave) y 2 (asteroide) con SPRITE. La OAM sombra, llena de EE.
- **P2, ninguna entidad.** Ninguna entidad viva. La OAM sombra, llena de EE.

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Prepares the whole shadow OAM (in RAM, so no hurry and no need for VBLANK):
;;  1. Fills it with zeros (Y = 0: every sprite hidden), so that
;;     the entities that no longer exist do not leave sprites behind.
;;  2. Puts shadow_cursor at the start of the shadow OAM.
;;  3. Prepares every entity with sys_prerender_one.
;; ❌ May modify A, BC, DE, HL
;; ⚠️ Use memset8 (clear) and man_entity_for_each
sys_prerender_update::
    ;; <<YOUR CODE>>
```

Ejercicio 8

Implementa `sys_render_update`

Espera al VBLANK y copia los primeros `SHADOW_OAM_SIZE` bytes de la `shadow_oam` a la OAM.
Esta rutina no sabe nada de entidades: solo copia.
Además marca el ritmo del juego: cada fotograma espera aquí (por la espera a VBLANK)

ⓘ Pasos (intenta antes sin mirarlos)

1. `call wait_vblank` (ya hecha).
2. `HL` = `shadow_oam`, `DE` = `OAM_START`, `B` = `SHADOW_OAM_SIZE`.
3. Salta a `memcpy8` : su `ret` vuelve a quien te llamó.

Consejos

- Son 32 bytes: caben de sobra en un VBLANK.
- Sin esperar al VBLANK, la copia puede perderse o dar sprites rotos, sobre todo en consola real.

Si funciona

- Los sprites se mueven con fluidez, sin parpadeos raros.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, copia la OAM sombra.** La OAM sombra tiene 00, 01, 02 ... 9F (160 bytes).
- **P2, solo copia 32 bytes.** La OAM sombra tiene 80, 81 ... (160 bytes). Antes, la OAM FE00-FE3F se llena con 77.

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Shows the sprites: waits for VBLANK and copies the first SHADOW_OAM_SIZE bytes of
;; the shadow OAM (the sprites of the entities) to the OAM, which is at OAM_START
($FE00).
;; The OAM can only be written during VBLANK (and with the screen off).
;; This routine knows nothing about entities: it is a plain copy.
;; ❌ May modify A, B, DE, HL
;; ⚠️ Use wait_vblank and memcpy8
sys_render_update::
    ;; <<YOUR CODE>>
```

Ejercicio 9

Completa `sys_collision_overlaps`

Comprueba si dos entidades se solapan: `DE` = entidad 1, `HL` = entidad 2.
Ocurre si en ***LOS DOS*** ejes (X e Y), la distancia entre ellas es menor que `HIT_SIZE` (ancho de colisión):

- `|Y1 - Y2| < HIT_SIZE`
- `|X1 - X2| < HIT_SIZE`

Devuelve en el carry: `C` = se solapan, `NC` = no.

✓ Optimización matemática

En este juego, todas las entidades son cuadradas y del mismo tamaño. Por eso, posición y ancho de colisión (`HIT_SIZE`) son suficientes. Es casi como si fueran círculos.

Si fueran rectangulares, la optimización es la misma, mientras tengan todas el mismo tamaño. Solo cambia que cada eje tiene su distancia (ancho y alto de colisión).

Con tamaños diferentes, hay que comprobar si sus intervalos se solapan en cada eje.

🖉 Distancia en valor absoluto `D = |U - V|`

`U - V` y `V - U` dan el mismo valor, pero positivo o negativo. `|U - V|` es el ***valor absoluto***: el valor positivo. Por tanto, `D = |U - V|` es la distancia entre `U` y `V` .

Para calcular este valor absoluto, tras hacer la resta (`ld a, U : sub V`) miramos el ***Carry***. Si hay Carry (`C`), la resta es negativa; invertimos a positivo, calculando su ***complemento a 2***: `cpl : inc a` , invertir bits + sumar 1.

Visualizando y entendiendo el solapamiento

Observa cómo estas entidades se solapan y sus posiciones (Y, X) y distancias en cada eje. Visualiza cómo la distancia de las posiciones se relaciona con el trozo solapado. Ejemplo con cuadrados de colisión de `lado = HIT_SIZE = 5` (**A**=entidad 1 · **B**=entidad 2 · **AB**=solapamiento):

Y↓ X→	48	49	50	51	52	53	54
50	·	·	A	A	A	A	A
51	·	·	A	A	A	A	A
52	B	B	AB	AB	AB	A	A
53	B	B	AB	AB	AB	A	A
54	B	B	AB	AB	AB	A	A
55	B	B	B	B	B	·	·
56	B	B	B	B	B	·	·

- `|dY| = 2 < lado` y `|dX| = 2 < lado` → Se solapan en 3 unidades (`lado - 2`)

Y↓ X→	50	51	52	53	54	55	56	57	58	59
50	A	A	A	A	A	B	B	B	B	B
51	A	A	A	A	A	B	B	B	B	B
52	A	A	A	A	A	B	B	B	B	B
53	A	A	A	A	A	B	B	B	B	B
54	A	A	A	A	A	B	B	B	B	B

- `|dY| = 0 < lado` y `|dX| = 5 >= lado` → No se solapan.
- Deben cumplirse ambas. No es suficiente con `|dY| < lado` .

ⓘ Pasos (intenta antes sin mirarlos)

1. El cálculo de `|dY|` ya está dado. Termina el eje Y: `cp HIT_SIZE` . Sin carry (distancia mayor o igual): `ret nc` .
2. Repite con la `X` : `inc de` / `inc hl` , resta y valor absoluto.

3. Termina con `cp HIT_SIZE` y `ret` : el carry es la respuesta.

Consejos

- Es el cálculo del Challenge 4 (`cpl` + `inc`), ahora en dos ejes.
- Si el primer eje falla, no hace falta mirar el segundo.

Si funciona

- Los disparos destruyen asteroides al tocarlos y la nave explota al chocar. Si pasa a distancia o no pasa, revisa este ejercicio y el 10.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, se solapan.** Entidad 1 en (Y=50, X=50) y entidad 2 en (Y=53, X=47). DE = entidad 1, HL = entidad 2. HIT_SIZE = 7.
- **P2, lejos en X.** Entidad 1 en (Y=50, X=50) y entidad 2 en (Y=50, X=57). DE = entidad 1, HL = entidad 2.

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Do two entities overlap? They do if, in BOTH axes, the distance
;; between their positions is less than HIT_SIZE: |Y1 - Y2| < HIT_SIZE
;; and |X1 - X2| < HIT_SIZE.
;; 📥 INPUT:   DE: entity 1,  HL: entity 2
;; 🔙 RETURNS (in the carry, as in Challenge 4):
;;    C,  they overlap
;;    NC, they do not
;; ❌ May modify A, DE, HL
sys_collision_overlaps::
    inc  de                     ;; \ DE -> Y1
    inc  hl                     ;; / HL -> Y2

    ld   a, [de]                ;; \ A = Y1 - Y2
    sub  [hl]                   ;; /

    jr   nc, .y_positive        ;; \ Positive: it is already |dY|
    cpl                         ;; | Negative: |dY| = -dY
    inc  a                      ;; / (cpl + inc: two's complement)

.y_positive

    ;; <<YOUR CODE>>
```

Ejercicio 10

Completa `sys_collision_one`

Comprueba colisión de la entidad candidata (en `DE`) contra la entidad sonda (en `collision_probe`).
La candidata (`DE`) debe cumplir tres condiciones:

- tener `COLLIDER`
- su `KIND` está en `collision_mask`
- se solapa con la sonda (`sys_collision_overlaps`)

Si choca, se libera (`man_entity_free`) y `collision_hits` suma 1.

Sistema de colisiones

`sys_collision_update` selecciona una entidad (`collision_probe`) y establece con qué puede colisionar (`collision_mask`). Después, usa `man_entity_for_each` para que llame a `sys_collision_one` una vez por cada entidad viva.
Así llega la ejecución a ti, con una entidad en `DE` para comprobar con la sonda.

Pasos (intenta antes sin mirarlos)

1. Filtro: sin `COLLIDER` , `ret` . (El filtro por `KIND` y máscara ya está dado.)
2. `HL` = puntero guardado en `collision_probe` .
3. Llama a `sys_collision_overlaps` (`DE` = candidata, `HL` = sonda). Protege `DE` con `push` y `pop` : la rutina lo modifica.
4. Sin solape (`NC`): `ret nc` . Con solape: `inc [hl]` sobre `collision_hits` y salta a `man_entity_free` (`DE` = candidata).

Consejos

- La máscara permite chocar con varios tipos a la vez: cada tipo es un bit de `KIND` .
- La nave y el disparo no se confunden: sus `KIND` no están en la máscara de los asteroides.

Si funciona

- Los asteroides desaparecen al alcanzarlos un disparo, y las monedas al recogerlas con la nave. El disparo atraviesa las monedas.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, la candidata choca.** Sonda = la nave (entidad 0, Y=144, X=84). Máscara = KIND_ENEMY. Candidata: asteroide en (Y=140, X=88), entidad 3.
- **P2, tipo que no está en la máscara.** Igual, pero la candidata es una moneda (KIND_BONUS) y la máscara solo tiene KIND_ENEMY.

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks ONE entity (the candidate) against the probe, which is in
;; collision_probe. If the candidate has the COLLIDER component, its
;; KIND is one of the kinds in collision_mask, and it overlaps the probe,
;; then it is freed and collision_hits is incremented.
;; 📥 INPUT: DE: candidate entity
;; ❌ May modify A, DE, HL
sys_collision_one::
    ;; <<YOUR CODE>>
    ld   hl, E_KIND             ;; \ HL = &candidate.KIND
    add  hl, de                 ;; |
    ld   a, [collision_mask]    ;; | A = collision_mask & candidate.KIND:
    and  [hl]                   ;; | is it a kind the probe collides with?
    ret  z                      ;; / No: ignore it

    ;; <<YOUR CODE>>
```

Ejercicio 11

Implementa `scene_menu_update`, `scene_gameover_update`

- `scene_menu_update`: si se acaba de pulsar START, cambia a `scene_game`.
- `scene_gameover_update`: si se acaba de pulsar START, vuelve a `scene_menu`.

Si no se ha pulsado START, no hacen nada.

Pasos (intenta antes sin mirarlos)

1. `A` = `input_pressed`: los botones pulsados en ***ESTE*** fotograma, no los que siguen pulsados.
2. Mira el bit de START (`and BTN_START`). Si no está: `ret z`.
3. `HL` = escena destino y salta a `scene_change`.

Consejos

- `input_held`: botones pulsados ahora. `input_pressed`: botones recién pulsados (***flanco*** de pulsación).
- Con `input_held`, al tocar START saltarían menú, juego y Game Over. Por eso usamos el flanco.
- `scene_change` apaga la pantalla, prepara la escena y la enciende. Salta con `jp`.

Si funciona

- START en el menú empieza la partida; START en Game Over vuelve al menú. A no hace nada.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, START en el menú.** Escena actual = menú. Se acaba de pulsar START. Llamamos a la actualización del menú.

- **P2, START en Game Over.** Escena actual = Game Over. Se acaba de pulsar START. Llamamos a la actualización de Game Over.

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Menu scene, every frame: if START has just been pressed, go to
;; the game scene (scene_game). If not, do nothing.
;; ⚠️ Use input_pressed (the "edge") and scene_change
scene_menu_update::
    ;; <<YOUR CODE>>
```

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Game over scene, every frame: if START has just been pressed, go to
;; the menu scene (scene_menu). If not, do nothing.
;; Each scene only updates its own things: this one does not touch the game.
scene_gameover_update::
    ;; <<YOUR CODE>>
```

Ejercicio 12

Completa `scene_game_update` , `game_handle_collisions`

- `scene_game_update` : un fotograma del juego. Llama ***en ORDEN*** a los sistemas: *check* de fin del juego + colisiones (justo tras renderizar) y después actualizar todo, paso a paso.
- `game_handle_collisions` : las reglas de colisión. Lanza 3 veces el sistema de colisiones, para cada tipo de colisión:

1. Disparo contra enemigos
2. Nave contra enemigos
3. Nave contra bonus

Pasos (intenta antes sin mirarlos)

1. `scene_game_update` : `is_ship_gone` y el cambio a Game Over ya están dados. Escribe las llamadas, en este orden: `game_handle_collisions` , `sys_input_update` , `game_handle_shooting` , `sys_spawn_update` , `sys_physics_update` , `sys_shot_update` , `sys_selfdestroy_update` y, al final, `sys_prerender_update` .
2. `game_handle_collisions` : con disparo (`sys_shot_get` ; `Z` = no hay), `B = KIND_ENEMY` y `sys_collision_update` . Si choca: `game_enemy_hit` . Sin disparo: salta a `.ship` .
3. Las pasadas de la nave contra asteroides y monedas ya están dadas.

Consejos

- `sys_collision_update` devuelve en `A` cuántas colisiones hubo y `Z` si ninguna: `call nz, rutina` solo llama si hubo.
- `is_ship_gone` va primero: ninguna entidad ha podido ocupar aún el hueco de la nave.
- **Colisiones después de renderizar**: el contacto se ve en pantalla antes de reaccionar.

- `sys_shot_update` va después de la física: ella mueve el disparo y puede liberarlo al salir de la pantalla.

Si funciona

- Se puede jugar una partida completa: disparar, destruir asteroides, chocar, ver Game Over y volver al menú.

Casos de prueba (P1 y P2)

Traza tu código a mano con ellos. En el proyecto, BGB compara CPU y MEM con lo esperado.

- **P1, un fotograma con un impacto.** La nave, un disparo (Y=60, X=100) y un asteroide encima (Y=62, X=100). Sin pulsar nada. spawn_timer = 100 (no sale otro).
- **P2, el disparo sale por arriba.** La nave y un disparo en la entidad 1 (Y = 2, X = 100). Sin pulsar nada. spawn_timer = 100 (no sale ningún asteroide).

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Game scene, every frame. First, the collisions; then, everything moves.
;; This way the collisions are checked with the positions that have just
;; been DRAWN, so the player sees the shot touch the asteroid (or the
;; asteroid touch the ship) before anything happens.
;;
;;  0. If the ship is gone (is_ship_gone: Z), it has finished exploding:
;;     go to the game over scene (scene_gameover).
;;     (This must be the first thing: nothing can have taken that slot yet.)
;;  1. game_handle_collisions: all the collisions of the game and what they cause.
;;  2. sys_input_update (sets VX from the D-pad), game_handle_shooting (A shoots),
;;     sys_spawn_update, sys_physics_update (it moves EVERYTHING: the ship, the
;;     asteroids and the shot), sys_shot_update (the physics may have freed the
;;     shot: it forgets it) and sys_selfdestroy_update.
;;  3. sys_prerender_update: prepares the sprites of how everything has ended
;;     up. The main loop copies them to the OAM in VBLANK.
scene_game_update::
    call is_ship_gone            ;; Z if the ship has gone
    jp   z, .game_over

    ;; <<YOUR CODE>>
.game_over
    ld   hl, scene_gameover
    jp   scene_change
```

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; All the collisions of the game, and what they cause. The system
;; (sys_collision_update) only checks ONE probe against all the entities:
;; the rules of the game say who is the probe, who it collides with and what happens.
;;  1. The shot (if there is one, sys_shot_get) against the asteroids (KIND_ENEMY).
;;     If it hits any, call game_enemy_hit.
;;  2. The ship (SHIP_ENTITY) against the asteroids. If it hits any:
;;     call game_ship_destroyed. The ship becomes an explosion and the
;;     game goes on until the explosion disappears.
;;  3. The ship against the bonus items (KIND_BONUS).
;;     If it catches any, call game_bonus_taken.
;; ⚠️ sys_collision_update returns Z (none) / NZ (at least one)
;; ❌ May modify A, BC, DE, HL
game_handle_collisions::
    ;; <<YOUR CODE>>
.ship
    ld   hl, SHIP_ENTITY        ;; \ HL = the ship (the probe)
    ld   b, KIND_ENEMY          ;; | B = it collides with asteroids
    call sys_collision_update   ;; | A = how many it has hit
    call nz, game_ship_destroyed ;; / Some: the ship becomes an explosion

    ld   hl, SHIP_ENTITY        ;; \ HL = the ship (the probe)
    ld   b, KIND_BONUS          ;; | B = it collides with bonus items
    call sys_collision_update   ;; | A = how many it has caught
    call nz, game_bonus_taken   ;; / Some: points

    ret
```

Y después

Explica **por qué** funciona cada ejercicio: qué hace cada bloque y qué pasaría sin él. Eso te permitirá adaptarlo en clase.

> 🔥 **Reflexión**
> Casi ninguna rutina sabe nada de las demás: solo cumple un contrato. ¿Qué te facilita eso al trabajar en equipo?